TUMAR Safety Platform

Autonomous Multi-Agent Customer Support System

KazMunayGas · Pilot: Zhetybai & Kalamkas · Target: 60 brigades (Oil Services Company)

Executive Summary

TUMAR is a KMG-deployed AI and computer-vision platform that monitors PPE compliance, dangerous-zone access, and hydraulic-key safety across camera networks at oil-field workovers. As the system scales to 60 brigades, inbound support volume — camera faults, false positives, connectivity failures, model errors — demands a 24/7 automated response pipeline. "Dark Factory" is a five-agent autonomous system that handles the full ticket lifecycle (receive → classify → resolve → notify → audit) with approximately 88% of tickets resolved without human intervention. All P1 (Priority 1) safety events and low-confidence classifications escalate to a duty engineer within five minutes.

Part 1: System Design

1.1 Agent Architecture

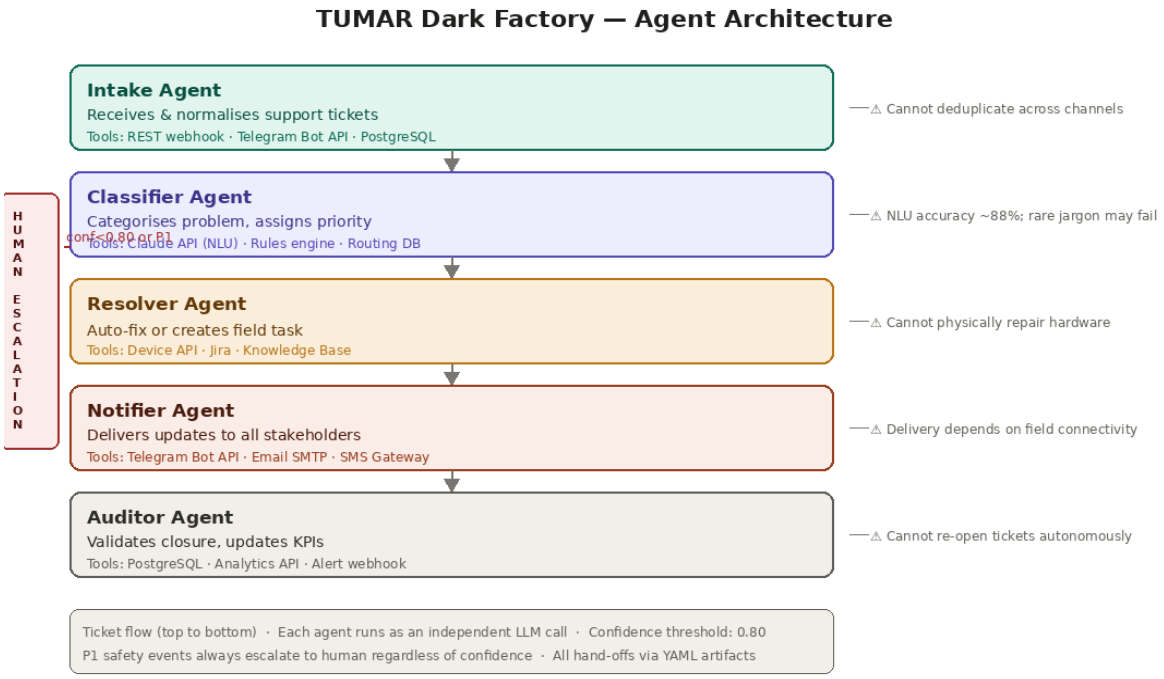

Figure 1 — Five-agent pipeline with tools, limitations, and escalation path

Agent	Role	Tools	Limitations
Intake	Receive & normalise tickets	REST webhook · Telegram Bot · PostgreSQL	No cross-channel dedup; no diagnosis
Classifier	Category · priority · team · confidence	Claude API (NLU) · Rules engine · Routing DB	~88% NLU accuracy; rare KZ jargon fails
Resolver	Remote auto-fix or Jira field task	Device API · Jira · Knowledge Base	Cannot physically repair; P1 needs human
Notifier	Updates to reporter & field team	Telegram · Email SMTP · SMS Gateway	Depends on field connectivity
Auditor	Validate closure · KPIs · anomalies	PostgreSQL · Analytics API · Alert webhook	Cannot re-open tickets autonomously

1.2 Workflow

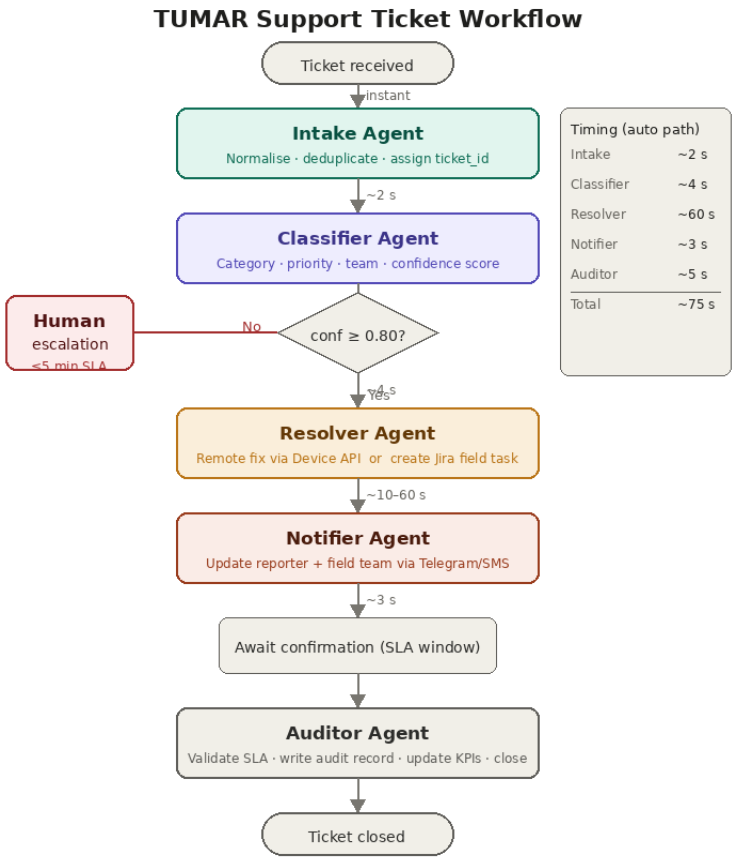

Figure 2 — End-to-end ticket lifecycle. Auto-path ~75 s total. SLA: P1 = 5 min, P2 = 30 min, P3/P4 = 4 h.

1.3 Artifact & Format Choice

YAML is used for all agent-to-agent hand-offs (≈35% fewer tokens than equivalent JSON). JSON is reserved for database writes requiring strict schema validation. Plain text for outbound notifications.

Artifact	Format	Justification	Producer → Consumer	~Tokens
Raw ticket	YAML	Flat key-value; 35% vs JSON	Intake → Classifier	60
Classified ticket	YAML	Routing data flat; same benefit	Classifier → Resolver	80
Resolution record	YAML	Nested log still compact	Resolver → Notifier	100
Notification payload	Plain text	Simple string; no structure	Notifier → Channels	40
Audit log / KPI event	JSON	DB & API require typed schema	Auditor → PostgreSQL/API	120

```
# YAML  (~60 tokens)              # JSON  (~85 tokens — same data, +42%)
ticket_id: t-9f3a                 {"ticket_id": "t-9f3a",
category:  camera                  "category":  "camera",
priority:  P2                      "priority":  "P2",
confidence: 0.93                   "confidence": 0.93 }
```

Part 2: System Prompts

Each agent operates with a tightly bounded system prompt specifying role, capabilities, I/O formats, and error-handling rules. The YAML I/O schemas are shown inline within each prompt entry.

Intake Agent

Role	Entry point. Receives raw inbound messages, parses them into canonical YAML tickets, assigns ticket_id (UUID-v4) and UTC timestamp. Does not classify or diagnose.
Can	Accept tickets via Telegram, email, REST webhook; extract site + reporter_id; strip PII beyond reporter_id
Cannot	Classify, diagnose, or contact field teams
Rules	(1) If site absent → site: UNKNOWN, missing_site: true. (2) If raw_text empty → status: INVALID, halt + log. (3) Output must be valid YAML within 3 s.

Input (YAML)
`source: telegram`
`reporter id: u 8821`
`site: Zhetybai-12`
`raw text: "Камера №3 не работает"`
	Output
	`ticket id: t-9f3a`
	`ts: 2025-06-01T08:14:00Z`
	`site: Zhetybai-12`
	`missing_site: false`

Errors / Standards	Source unrecognised → source: UNKNOWN, continue. All fails → dead-letter queue. Standard: YAML must parse on every execution.

Classifier Agent

Role	Decision brain. Categorises ticket into 8 categories, assigns priority P1–P4, routes to responsible team, outputs confidence 0–1. Does not act on devices.
Can	Classify (camera/angle/compute/network/model_error/alarm/wifi/false_positive); assign P1–P4; route to IT_field / AI_team / safety_officer
Cannot	Take device actions; communicate with reporters
Rules	(1) NLU + keyword rules; if conf < 0.80 → escalate_human: true. (2) Safety-zone/alarm keyword → P1 immediately. (3) Unknown language → flag + escalate.

Input (YAML)
`ticket_id: t-9f3a`
`site: Zhetybai-12`
`raw_text: "Камера №3 не работает"`
	Output
	`category: camera`
	`priority: P2`
	`team: IT_field`
	`confidence: 0.93`
	`escalate human: false`

Errors / Standards	NLU fails → retry once; if again fails → conf: 0.0, escalate. Standard: never hard-code confidence 1.0; P1 never auto-closed.

Resolver Agent

Role	Action executor. Attempts remote fix via Device API or creates a structured Jira field task using Knowledge Base steps. Logs every API call and HTTP response.
Can	Send reboot/restart via Device API; create Jira tasks with KB steps + ETA; query device status (ping, last heartbeat)
Cannot	Physically replace hardware; approve safety overrides without human

Rules	(1) Look up KB for category. (2) If remote fix exists → attempt; log response code. (3) If fails → Jira task + manual_required. (4) P1 → task AND page safety_officer.

Input (YAML)
`ticket_id: t-9f3a`
`category: camera`
`priority: P2`
`team: IT_field`
	Output
	`action_taken: reboot_camera_service`
	`result: success`
	`jira_task: null`
	`resolved_at: 2025-06-01T08:17:00Z`

Errors / Standards	Device API unreachable → skip, create Jira, set manual_required. KB miss → task with description only, flag kb_miss: true. Standard: result: success only on HTTP 200 + device confirmation.

Notifier Agent

Role	Communication layer. Sends status updates to reporter and field team on the appropriate channel, in the reporter's preferred language (default: Russian).
Can	Send Telegram to reporter/team; email supervisors for P1/P2; SMS fallback for offline sites
Cannot	Modify resolution records; expose internal metadata (confidence, Jira IDs) to recipients
Rules	(1) result: success → send resolved message. (2) manual_required → send task + ETA to team and reporter. (3) P1 → also email safety_officer. (4) Confirm delivery before logging status: delivered.

Input (YAML)
`ticket_id: t-9f3a`
`result: success`
`team: IT_field`
`reporter_id: u_8821`
	Output
	`notifications:`
	`  - channel: telegram`
	`  recipient: u_8821`
	`  sent_at: 2025-06-01T08:17:45Z`
	`  status: delivered`

Errors / Standards	Telegram fails → retry 2×; then SMS. All channels fail → log critical_notification_failure, alert Auditor. Standard: delivery must be confirmed before logging.

Auditor Agent

Role	Quality gate and analytics recorder. Validates resolution completeness, calculates SLA compliance, writes immutable audit records to PostgreSQL, flags anomalies.
Can	Query resolution + notification logs; calculate SLA (P1: 5 min, P2: 30 min, P3/P4: 4 h); push KPI events to Analytics API + PostgreSQL
Cannot	Re-open or modify closed tickets; contact reporters
Rules	(1) If notification not delivered → flag closure_blocked, do not close. (2) false_positive > 5/site/day → alert AI_team. (3) DB write fails → retry 3× exponential backoff then local file queue. (4) P1 SLA breach → alert management.

Input (YAML)
`ticket_id: t-9f3a`
`resolved_at: 2025-06-01T08:17:00Z`
`notifications: [{status: delivered}]`
`priority: P2`
	Output
	`{"ticket_id":"t-9f3a",`
	`"closed_at":"2025-06-01T08:18:00Z",`
	`"sla_met":true,"priority":"P2",`
	`"false_positive":false}`

Errors / Standards	All DB retries fail → local file queue for sync. Standard: one immutable audit record per ticket; all timestamps UTC.

Part 3: Hand-offs & Control

3.1 Decision Logic

From → To	Trigger	Data Passed	Validation	If Fails
Intake → Classifier	ticket_id created	Full ticket YAML	YAML parses; site not null	Retry once; dead-letter queue
Classifier → Resolver	confidence ≥ 0.80	Classified ticket	category in allowed list	Escalate to human
Classifier → Human	conf < 0.80 or P1	Ticket + scores	Ack within 5 min	Re-alert ×3 every 5 min
Resolver → Notifier	action_taken set	Resolution record	result field present	Log; notify 'pending' anyway
Notifier → Auditor	All notifications sent	Notification log	≥ 1 delivery confirmed	Flag closure_blocked
Auditor → Close	SLA check passed	Audit record	DB write successful	Retry; alert on P1 breach

3.2 Human Escalation Scenarios

Scenario	Trigger	Context Sent	Response SLA
Low confidence	conf < 0.80	Full ticket + top-3 categories	5 min
Critical safety (P1)	P1 any category	Ticket + site + camera feed link	2 min
Repeated false positives	> 5/site/day	Aggregated false_positive log	30 min
Notification total failure	All channels failed	ticket_id + reporter contact	Immediate

3.3 Monitoring Thresholds

Metric	Warning	Critical	Action
Classifier accuracy (daily 50-ticket audit)	< 90%	< 80%	Retrain; expand rules
P1 response time	≥ 3 min	≥ 5 min	Page duty engineer
False positive rate / site / day	10%	25%	Alert AI team; model review
Notification delivery rate	< 95%	< 85%	Switch to fallback channel
DB write success rate	< 99%	< 95%	Activate local file queue
SLA breach rate (P1+P2 / week)	5%	15%	Management escalation

Part 4: Economics & Limitations

4.1 Token Analysis

Assumptions: Claude Sonnet 4 at $5.00 / 1M input tokens, $15.00 / 1M output tokens. Volume: 1,000 workflows/day × 30 days. ~12% escalated tickets add ~2× context overhead.

Agent	Sys. Prompt	Input	Output	Total / exec
Intake Agent	320	80	60	460
Classifier Agent	480	120	80	680
Resolver Agent	520	200	120	840
Notifier Agent	380	140	80	600
Auditor Agent	440	160	90	690
Total / workflow	2 140	700	430	**3 270**

Cost Item	Calculation	Value
Monthly workflows	1,000 × 30 days	30,000
Monthly input tokens (~65%)	63,765,000 × $5 / 1M	$318.83
Monthly output tokens (~35%)	34,335,000 × $15 / 1M	$515.03
Base LLM cost	Input + Output	$833.86
Escalation overhead (12% tickets, 2× tokens)	~9.9M extra tokens	≈ $125
Total estimated monthly cost		**≈ $960**
Cost per ticket	$960 ÷ 30,000	**$0.032**

4.2 Drawbacks & Mitigations

Category	Limitation	Impact	Mitigation
Technical	NLU accuracy ceiling (~88%) on short RU/KZ field messages	12% misclassified → wrong team	Hybrid rules + NLU; weekly retraining on human corrections
Technical	Device API latency at remote satellite/4G sites	Resolver timeouts → false manual_required	Async 60 s polling + local edge agent for on-site reboots
Technical	Kazakh under-represented in training data	KZ tickets less accurate than Russian	Dedicated KZ preprocessing + domain keyword dictionary

Category	Limitation	Impact	Mitigation
Operational	Notification failure in offline zones	Reporter gets no update	SMS fallback + site LCD status board
Operational	Alert fatigue from repeated P1 false-positive escalations	Engineers ignore alerts	5-alert max before queue; false-positive suppression
Economic	60-brigade rollout = 6× volume → ~$5,800/month LLM cost	Budget overrun	Route non-NLU steps to rules engine; negotiate enterprise pricing

4.3 Reflection

Question	Answer
Single biggest limitation?	NLU on short noisy RU/KZ messages with domain jargon. Misclassification at step 2 corrupts every downstream action — wrong team, wrong resolution, wrong notification.
If 2× budget, improve first?	Fine-tune a Kazakh/Russian oil-industry NLU model with real-time feedback from human corrections. Reducing escalation rate 12% → 4% yields highest downstream ROI.
Catastrophic failure point?	Notifier total channel failure during a P1 safety event. If hydraulic-key danger is detected but no alert reaches the safety officer — this is a physical safety incident, not a support ticket failure.
% truly dark?	~85–88% of tickets by volume run fully autonomously. P1 safety events are never dark — human confirmation mandatory for closure.
Cost-effective vs humans?	$0.032/ticket vs ~$8–15 for a human agent covering 60 brigades 24/7 ≈ 250–470× cheaper. Holds only if classifier accuracy ≥ 85% and escalation paths stay fast.